

Revisión Papers Semana 2.

Nicolás Parra

Software Bill of Materials Adoption: A Mining Study from GitHub

¿Cuál es el problema que se está resolviendo?

El artículo aborda el problema de la baja adopción de las Software Bill of Materials (SBOMs) en proyectos de software de código abierto, a pesar de su creciente importancia para la seguridad y la trazabilidad en la cadena de suministro de software. Aunque gobiernos como el de Estados Unidos y la Unión Europea han comenzado a exigirlos o recomendarlos, su uso real en proyectos públicos es aún limitado.

¿Cuál es la nueva idea que están proponiendo?

Se propone un análisis empírico, usando minería de repositorios en GitHub, para medir cuántos proyectos adoptan herramientas SBOM basadas en SPDX y CycloneDX. Se estudian 186 repositorios, sus patrones de adopción, actualización y distribución de SBOMs, así como tendencias geográficas. A diferencia de estudios previos cualitativos, este enfoque es totalmente automatizado y cuantitativo.

Puntos positivos

1. Enfoque automatizado: Uso de herramientas como GitHub Dependency Graph y PyDriller para obtener datos objetivos.
2. Cobertura amplia: Análisis de múltiples dimensiones: estándares, regiones, frecuencia de actualización, integración en CI/CD.
3. Relevancia: Trata un tema actual y alineado con políticas públicas de seguridad informática.

Puntos negativos

1. Alcance limitado: Solo considera GitHub y dos estándares de SBOM.
2. Sin contraste cualitativo: No incluye opiniones o experiencias de desarrolladores.

3. No analiza impacto práctico: No se mide si los SBOMs realmente ayudan frente a vulnerabilidades.

Trabajo futuro

Investigar si proyectos con SBOMs responden más rápido a vulnerabilidades. También, crear herramientas que alerten sobre SBOMs desactualizados o ausentes, integradas a CI/CD.

Calificación: 5 / 5

El artículo presenta un enfoque riguroso, relevante y bien ejecutado para evaluar la adopción de SBOMs en proyectos de código abierto. Se apoya en datos empíricos y herramientas automatizadas, lo que le da objetividad y escalabilidad. Además, aborda un tema de alto impacto actual en ciberseguridad.

Puntos de discusión

1. ¿Deberían plataformas como GitHub impulsar activamente el uso de SBOMs?
2. ¿La regulación es suficiente incentivo para que la comunidad open source adopte SBOMs?

Otros comentarios

El dataset generado es un buen recurso para futuras investigaciones. El enfoque puede servir de base para guías de buenas prácticas en desarrollo seguro.

On the Accuracy of GitHub's Dependency Graph

¿Cuál es el problema que se está resolviendo?

El artículo aborda la precisión del dependency graph de GitHub, una herramienta fundamental para herramientas como Dependabot o la generación de SBOMs. Inexactitudes en este grafo pueden comprometer tanto la seguridad como la confiabilidad de investigaciones basadas en él.

¿Cuál es la nueva idea que están proponiendo?

Se propone un estudio empírico que evalúa la precisión del grafo de dependencias en 635 proyectos de código abierto en Java y Python. Se aplican tres análisis:

- Backward: evalúa si las dependencias reportadas incluyen correctamente al repositorio objetivo.
- Forward: verifica si los dependientes están correctamente listados.
- Manifest/lock: compara lo que reporta el grafo frente a los archivos reales de configuración (pom.xml, requirements.txt, etc.).

El estudio identifica numerosos errores, especialmente en proyectos Java, y sugiere que el grafo es más preciso para proyectos Python. También se discuten causas comunes de error (como enlaces rotos, versiones no estandarizadas o archivos ausentes) y se proponen implicaciones para desarrolladores, investigadores y la propia plataforma GitHub.

Puntos positivos

1. Diseño metodológico sólido: El uso de muestras estadísticas representativas con 95% de confianza da robustez a los resultados.
2. Trato riguroso de los datos: Combinan análisis cuantitativo y cualitativo, ilustrando causas reales de error con ejemplos.
3. Implicaciones prácticas claras: Identifican cómo los errores afectan a herramientas populares y dan recomendaciones para usuarios e investigadores.

Puntos negativos

1. Lenguaje y herramientas limitados: Solo se evaluaron Java y Python, excluyendo otros ecosistemas importantes como JavaScript o Rust.
2. Dependencia de scraping: La falta de APIs oficiales obligó al uso de web scraping, lo que puede afectar la reproducibilidad futura.
3. Análisis temporal único: Se usó una única extracción de datos (abril 2023), por lo que no se mide si GitHub ha mejorado desde entonces.

Trabajo futuro

Se podría replicar el estudio periódicamente para ver si GitHub mejora con el tiempo. También sería útil extenderlo a otros lenguajes o integrar métricas de impacto, como cuánto afecta cada tipo de error a las alertas de seguridad generadas por Dependabot. Otra idea sería desarrollar herramientas para validar automáticamente la información del grafo versus los archivos reales del repositorio.

Calificación: 4 / 5

El artículo es riguroso, práctico y oportuno. Solo le faltó un alcance más amplio en lenguajes y una perspectiva temporal para ser sobresaliente. Aun así, sus aportes son valiosos para la comunidad técnica y académica.

Puntos de discusión

1. ¿Deberían los desarrolladores confiar ciegamente en el dependency graph para seguridad y SBOMs?
2. ¿Qué mejoras podrían implementar GitHub u otros actores para validar la precisión del grafo automáticamente?

Otros comentarios

El artículo pone en evidencia que incluso herramientas ampliamente adoptadas como el dependency graph pueden tener limitaciones importantes, y sugiere que los desarrolladores no deben asumir que sus resultados son siempre correctos.

Software Bills of Materials Are Required. Are We There Yet?

¿Cuál es el problema que se está resolviendo?

La adopción de Software Bills of Materials (SBOMs) es obligatoria para proveedores que trabajan con el gobierno de EE. UU., pero su implementación en la industria es baja. A pesar de su potencial para mejorar la seguridad del software, existen múltiples obstáculos que impiden su uso generalizado.

¿Cuál es la nueva idea que están proponiendo?

El artículo realiza una revisión de literatura gris — blogs, informes, entrevistas— para identificar los principales beneficios y desafíos de adoptar SBOMs. Analiza 200 artículos, filtrando 62 relevantes, y clasifica los hallazgos en cinco beneficios y cinco desafíos. El objetivo es ayudar a líderes gubernamentales e industriales a crear planes de acción que impulsen una adopción efectiva. Entre los beneficios destacan:

1. Gestión de dependencias: claridad sobre los componentes de software utilizados.
2. Gestión de vulnerabilidades: respuesta más rápida ante amenazas como Log4Shell.
3. Gestión de riesgos y licencias: facilita la evaluación y mitigación de riesgos y cumplimiento legal.
4. Ventaja competitiva: quienes ofrecen SBOMs pueden acceder a mercados más exigentes.

Por otro lado, los desafíos principales incluyen la falta de herramientas maduras, problemas de interoperabilidad entre estándares, dudas sobre el valor real de los SBOMs, el tiempo requerido para mantenerlos, y preocupaciones por los riesgos de transparencia.

Puntos positivos

1. Metodología práctica y actual: usar literatura gris permite reflejar la realidad industrial, no solo la académica.
2. Clasificación clara de beneficios y retos: estructuración útil para tomadores de decisiones.
3. Enfoque en impacto real: se vincula con casos como Log4Shell y requisitos gubernamentales actuales.

Puntos negativos

1. Falta de validación empírica: no se incluyen estudios de caso ni datos medibles de impacto real en seguridad.
2. Dependencia en fuentes no revisadas por pares: la literatura gris puede estar sesgada o desactualizada.
3. Perspectiva centrada en EE. UU.: se echa en falta un análisis más global o multirregional.

Trabajo futuro

Sería útil estudiar empíricamente el impacto de SBOMs en la respuesta a vulnerabilidades (por ejemplo, medir el tiempo de remediación con y sin SBOM). También se podría desarrollar una plataforma estándar interoperable para compartir SBOMs entre proveedores y clientes, automatizando su uso dentro del ciclo de vida del software.

Calificación: 4 / 5

El artículo es relevante, oportuno y bien estructurado, pero se beneficiaría de incluir datos empíricos y una visión más internacional. Aporta mucho al debate actual sobre seguridad en la cadena de suministro.

Puntos de discusión

- ¿Cómo se puede equilibrar la transparencia de los SBOMs con la protección de la propiedad intelectual?
- ¿Deberían los gobiernos y grandes compradores exigir no solo SBOMs, sino también su consumo y validación por parte de los clientes?